

Data Warehousing

Lecture- 5: DW Modelling (Types of Dimensions)

Ali Zidane ElQutaany

Types of Dimensions

- **TEN Types of Dimension Tables:**
 1. Slowly Changing Dimensions.
 2. Conformed Dimensions (Distorted Dimension)
 3. Degenerate Dimensions.
 4. Junk Dimensions.
 5. Role Playing Dimensions.
 6. Static Dimensions (Fixed Dimensions)
 7. Shrunk Dimensions (Foreign Dimensions Manager).
 8. Stacked Dimensions.
 9. Deferred Dimension (Sensible Dimension).
 10. Rapidly Changing Dimensions.

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

1. **Type 0 - The passive method** (some values are overwritten (**Updated ONES**), and others remain the same). Some records are updated based on CDC (Capture Data Change)
2. **Type 1 - Overwriting the old value.** In this method no history of dimension changes is kept. The old dimension value is simply overwritten by the new one. This type is easy to maintain and is often use for data which changes are caused by processing corrections(e.g., removal special characters, correcting spelling errors).

Before the change:

Customer_ID	Customer_Name	Customer_Type
1	Cust_1	Corporate

After the change:

Customer_ID	Customer_Name	Customer_Type
1	Cust_1	Retail

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

- 3. **Type 2** - Creating a new additional record:

- In this methodology all history of dimension changes is kept.
 - You capture attribute change by adding a new row with a new surrogate key to the dimension table. Both the prior and new rows contain as attributes the natural key(or another durable identifier).
 - Also, **Effective Date** and 'current flag' columns are used in this method. There could be only one record with current indicator set to 'Y'. For 'effective date' columns, i.e., Start Date and End Date, the End Date for current record usually is set to value 9999-12-31.
 - Introducing changes to the dimensional model in type 2 could be very expensive database operation so it is not recommended to use it in dimensions where a new attribute could be added in the future.

Slowly Changing Dimensions

- Slowly Changing Dimensions: Type 2 -** Creating a new additional record:

Before the change:

Customer_ID	Customer_Name	Customer_Type	Start_Date	End_Date	Current_Flag
1	Cust_1	Corporate	22-07-2010	31-12-9999	Y

After the change:

Customer_ID	Customer_Name	Customer_Type	Start_Date	End_Date	Current_Flag
1	Cust_1	Corporate	22-07-2010	17-05-2012	N
2	Cust_1	Retail	18-05-2012	31-12-9999	Y

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

- 4. **Type 3** - Adding a new column:

- In this type usually only the current and previous value of dimension is kept in the database. The new value is loaded into 'current/new' column and the old one into 'old/previous' column.
 - The history is limited to the number of column created for storing historical data. This is the least commonly needed technique.

Slowly Changing Dimensions

- **Slowly Changing Dimensions: Type 3** - Adding a new column:

Before the change:

Customer_ID	Customer_Name	Current_Type	Previous_Type
1	Cust_1	Corporate	Corporate

After the change:

Customer_ID	Customer_Name	Current_Type	Previous_Type
1	Cust_1	Retail	Corporate

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

- 5. **Type 4** - Using historical table.

- The 'main' dimension table keeps only the current data e.g. customer and customer history tables.
 - In this method a separate historical table is used to track all dimension's attribute historical changes for each of the dimension.

Slowly Changing Dimensions

- **Slowly Changing Dimensions: Type 4** - Using historical table.

Current table:

Customer_ID	Customer_Name	Customer_Type
1	Cust_1	Corporate

Historical table:

Customer_ID	Customer_Name	Customer_Type	Start_Date	End_Date
1	Cust_1	Retail	01-01-2010	21-07-2010
1	Cust_1	Oher	22-07-2010	17-05-2012
1	Cust_1	Corporate	18-05-2012	31-12-9999

Slowly Changing Dimensions

- **Slowly Changing Dimensions:**

Type 6 - Combine approaches of types 1,2,3 ($1+2+3=6$). In this type we have in dimension table such additional columns as:

- **Current Type** - for keeping current value of the attribute. All history records for given item of attribute have the same current value.
- **Historical Type** - for keeping historical value of the attribute. All history records for given item of attribute could have different values.
- **StartDate**- for keeping start date of 'effective date' of attribute's history.
- **End Date**- for keeping end date of 'effective date' of attribute's history.
- **Current Flag**- for keeping information about the most recent record.
- In this method to capture attribute change we add a new record as in type 2. The **Current Type** information is overwritten with the new one as in type 1. We store the history in a **Historical Column** as in type 3.

Slowly Changing Dimensions

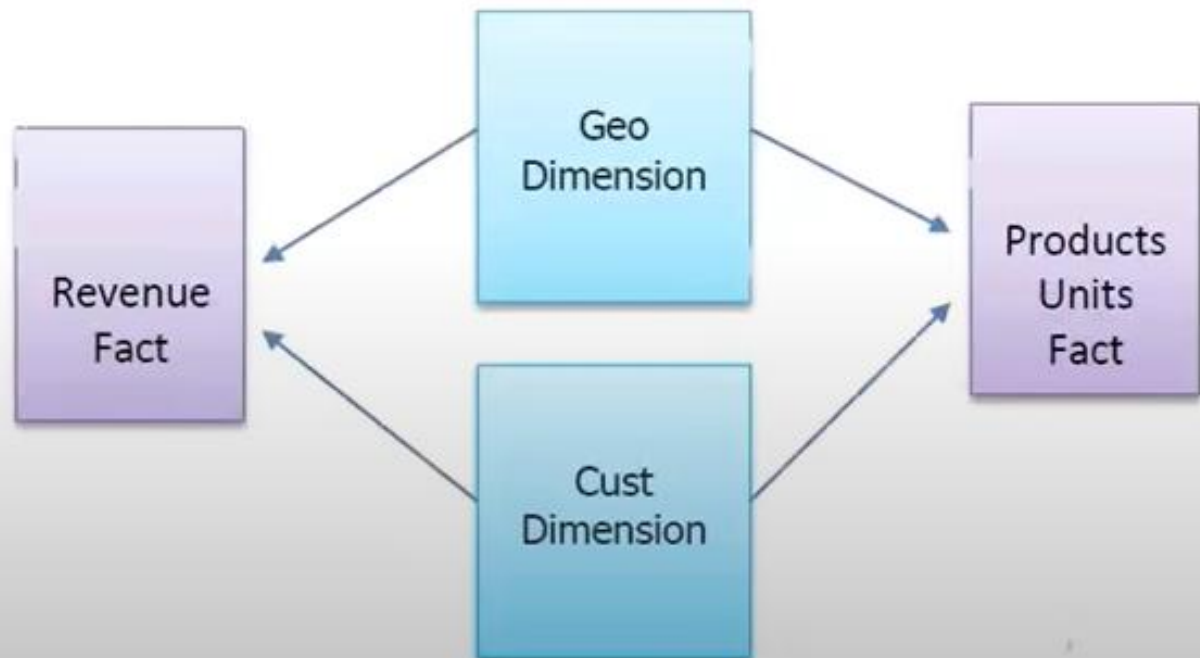
- **Slowly Changing Dimensions: Type 6**- Combine all types

Customer_ID	Customer_Name	Current_Type	Historical_Type	Start_Date	End_Date	Current_Flag
1	Cust_1	Corporate	Retail	01-01-2010	21-07-2010	N
2	Cust_1	Corporate	Other	22-07-2010	17-05-2012	N
3	Cust_1	Corporate	Corporate	18-05-2012	31-12-9999	Y

1. ~~Slowly Changing Dimensions.~~
2. **CONFORMED DIMENSIONS.**
3. Degenerate Dimensions.
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

CONFORMED DIMENSIONS (DISTORTED DIMENSIONS)

- In the following example; Geography and Customer Dimensions are being shared by 2 Fact Tables.
 - ✓ Revenue Fact Table stores the revenue generated by both Geography and the Customer.
 - ✓ Product Units Fact Table stores number of units sold in Geography to a Customer.
- Geography and Customer Dimensions are called **CONFORMED DIMENSIONS.**



1. Slowly Changing Dimensions.
2. Conformed Dimensions.
3. **DEGENERATE DIMENSIONS.**
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

DEGENERATE DIMENSIONS

- Sometimes a dimension is defined that has no content except for its **PRIMARY KEY**.
- A dimension which has ONLY primary key attribute and this attribute can be stored in a fact table and has no associated dimension table is called **DEGENERATE DIMENSION**.
- This **DEGENERATE DIMENSION** is placed in the fact table with the explicit acknowledgment that there is no associated dimension table.
- Degenerate dimensions are most common with **Transaction and Accumulating Snapshot Fact Tables**.

1. Slowly Changing Dimensions.
2. Conformed Dimensions.
3. **DEGENERATE DIMENSIONS.**
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

- As an example:

- ✓ The table below is fact table with Quantity and Amount as ADDITIVE measures.
- ✓ ProductId comes from Product Dimension.
- ✓ Sometimes, business would want to know the Quantity of products and Amount per **INVOICE**. In this case **InvoiceNumber** is a mandatory attribute in the Fact Table.
- ✓ Invoice Dimension is a dimension with ONLY one attribute InvoiceNumber.
- ✓ In this case; Invoice dimension is DEGENERATED (Ignored) and add the individual attribute directly to the Fact Table.

Invoice Number	ProductId	Quantity	Amount
04182022-1	123	4	200
04182022-1	341	3	100
04192022	124	20	400
04202022	178	1	300

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. **JUNK DIMENSIONS.**

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. Deferred Dimension.

10. Rapidly Changing Dimensions.

JUNK DIMENSIONS

(FOREIGN DIMENSIONS MANAGER)

- Assume the ATM log is stored in the following form, where there are multiple flags which are set when a transaction T takes place.
 - ✓ If the report generated from the DW shows that most of the transactions failed because there was no cash within the ATM. This could suggest that the cash delivery system is not efficient and corrective steps are needed.
 - ✓ These flags **DO NOT** integrate with the common dimensions like Customer, Product, ... ETC. As these flags are related to the system itself and not to a customer or product.

Successful Tran	Failed	No Cash	Invalid Pin	No Balance
1	0	0	0	0
0	1	1	0	0
0	1	0	1	0
0	1	0	0	1

JUNK DIMENSIONS

(FOREIGN DIMENSIONS MANAGER)

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. **JUNK DIMENSIONS.**
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

- ✓ Accommodating all these flags into a fact table will make the number of fields in the fact table very huge and result in performance degradation.
- ✓ Also, every field in the fact table will have a corresponding dimension table (even if it was a Degenerate Dimension). Hence if we have 20 flags, then we need to have 20 different dimensions.
- ✓ Instead of building one dimension for each of these flags, we can combine all the flags and form one single dimension called **JUNK DIMENSION** as below:

JunkDimId	Successful Tran	Failed	No Cash	Invalid Pin	No Balance
1	1	0	0	0	0
2	0	1	1	0	0
3	0	1	0	1	0
4	0	1	0	0	1

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. **JUNK DIMENSIONS.**

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. Deferred Dimension.

10. Rapidly Changing Dimensions.

JUNK DIMENSIONS (FOREIGN DIMENSIONS MANAGER)

✓ For every combination of these flags, there is a JunkDimId which will be used inside the Fact Table.

✓ This will achieve **THREE** objectives:

1. Reduces the number of fields within the fact table.

2. Saves from creating one dimension for each of the flags.

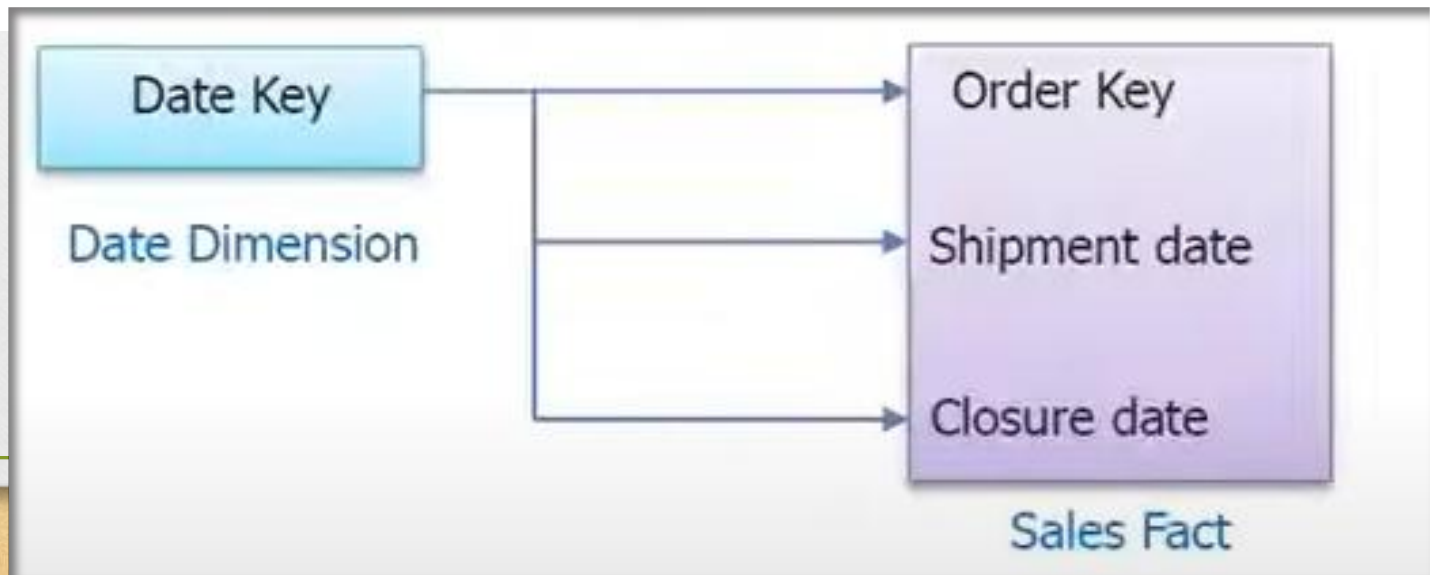
3. Increases the performance.

JunkDimId	Successful Tran	Failed	No Cash	Invalid Pin	No Balance
1	1	0	0	0	0
2	0	1	1	0	0
3	0	1	0	1	0
4	0	1	0	0	1

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. **ROLE PLAYING DIMENSIONS.**
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

ROLE PLAYING DIMENSIONS.

- **Role Playing Dimension** is the dimension which may has **MULTIPLE VALID** relationships with the Fact Table.
- As an example, the date dimension can be used to fetch **Order Placed Date**, **Order Shipment Date** or **Order Closure Date**.
- As can be seen below, there are multiple keys in the fact table pointing to the same dimension.



1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. **ROLE PLAYING DIMENSIONS.**

6. Static Dimensions.

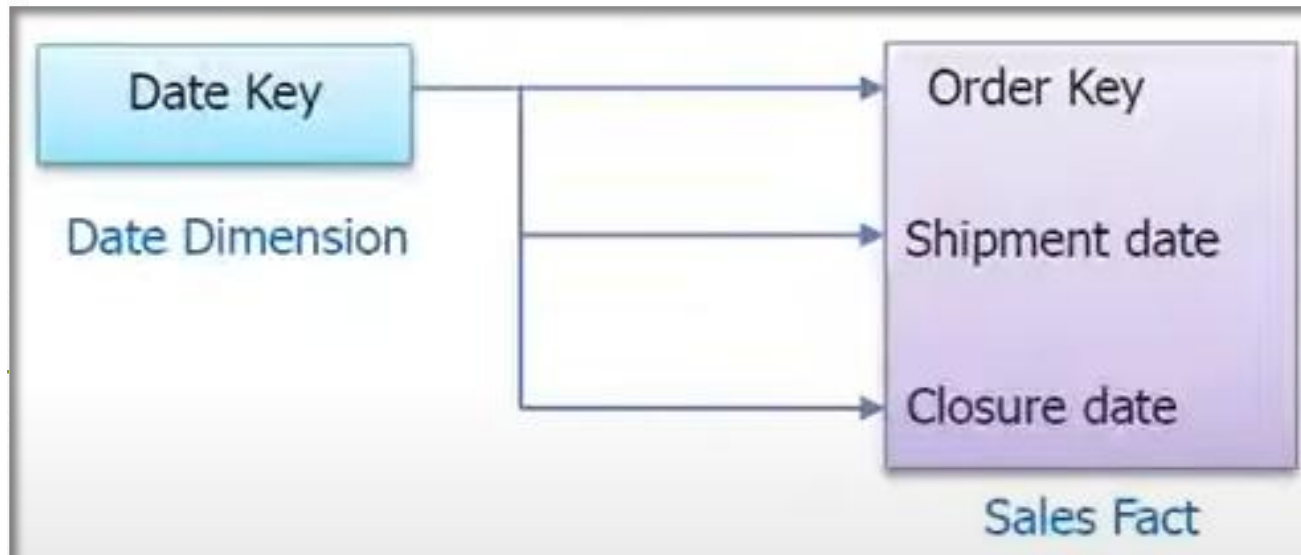
7. Shrunk Dimensions.

8. Stacked Dimensions.

9. Deferred Dimension.

10. Rapidly Changing Dimensions.

ROLE PLAYING DIMENSIONS.



- ✓ In this case we do not have to create 3 Date Dimensions.
- ✓ All the THREE Date References refer to the same Date Dimension.
- ✓ The Date Dimension is stored only once, processed once and accessed THREE times. This saves time and memory space.

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. **STATIC DIMENSIONS.**
7. ~~Shrunk Dimensions.~~
8. ~~Stacked Dimensions.~~
9. ~~Deferred Dimension.~~
10. ~~Rapidly Changing Dimensions.~~

STATIC DIMENSIONS. (FIXED DIMENSION)

- **Static Dimensions** are those dimensions which are not extracted from the original data source but are created within the context of the data warehouse.
- **Static Dimensions** can be:
 - ✓ **Loaded Manually** - for example with status codes.
 - ✓ **Generated By a Procedure**, such as a date or time dimension
- **As an example, the Date, Geography or Location Dimensions.**

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. **SHRUNKEN DIMENSIONS.**
8. Stacked Dimensions.
9. Deferred Dimension.
10. Rapidly Changing Dimensions.

SHRUNKEN DIMENSIONS.

- A **Shrunken Dimension** is a subset of another dimension.
- For example, the orders fact table may include a foreign key for product, but the target fact table may include a foreign key only for Product Category, which is in the product table, but much less granular.
- Creating a smaller dimension table, with Product Category as its primary key, is one way of dealing with this situation of heterogeneous grain.
- If the product dimension is snowflaked, there is probably already a separate table for Product Category, which can serve as the **shrunk dimension**

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. ~~Shrunk Dimensions.~~
8. **STACKED DIMENSIONS.**
9. ~~Deferred Dimension.~~
10. ~~Rapidly Changing Dimensions.~~

STACKED DIMENSIONS

- A **Stacked Dimension** is used where two or more dimensions are combined, and their attributes are Fact Table Properties and not dimensions properties.
- We see many types and status columns: Product Type, Customer Status, Store Type, Security Type, Security Class, Broker Type, etc. All the columns store to their respective dimensions because they are the properties of the dimension.

TransactionTypeKey	TransactionType	Category
0	Unknown	Unknown
1	Direct sale	Transaction type
2	Refund	Transaction type
3	Purchase	Transaction type
4	eCommerce	Transaction type
5	Down payment	Payment type
6	Full delivery	Payment type
7	On delivery	Payment type

- ~~1. Slowly Changing Dimensions.~~
- ~~2. Conformed Dimensions.~~
- ~~3. Degenerate Dimensions.~~
- ~~4. Junk Dimensions.~~
- ~~5. Role Playing Dimensions.~~
- ~~6. Static Dimensions.~~
- ~~7. Shrunk Dimensions.~~
- ~~8. Stacked Dimensions.~~
9. **DEFERRED DIMENSION.**
10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

- When loading a fact record, a dimension record may not be ready yet.
- One solution is to generate a surrogate key with null for all the other attributes. This should technically be called an inferred member but is often called an inferred dimension.
- It is technically called an inferior member but is often called as sensible dimension.
- A fact table that references dimension members that are not yet loaded contains inferred members.
- Data for inferred members can be updated rather than created when it is loaded, so you do not need to create a new record.
- **A late-arriving dimension** (sometimes called **early-arriving facts**) occurs when the dimension data arrives in the data warehouse later than the fact data that references it.

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. ~~Shrunk Dimensions.~~
8. ~~Stacked Dimensions.~~
9. **DEFERRED DIMENSION.**
10. **Rapidly Changing Dimensions.**

DEFERRED DIMENSION

INFERRED DIMENSIONS

Promo Code PK	Promo Code	Promotion Type	Discount Type	Inferred
1	P1	Marketing	Cashback	0
2	P2	Employee	Instant	0
3	Pspecial	Unknown	Unknown	1

TicketId	Transaction Date	Promo Code PK	Quantity
T1	04172022	1	100
T2	04182022	2	200
T3	04202022	3	700

1. Slowly Changing Dimensions.

2. Conformed Dimensions.

3. Degenerate Dimensions.

4. Junk Dimensions.

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

Employee

Team

Transaction System

OLAP System

DEFERRED DIMENSION

INFERRED DIMENSIONS

EmpId	Name	Gender	Salary
1	Ahmed	Male	10000
22	Maye	Female	12000

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12

Id	ProductId	CustId	EmpId	Amount
1	P1	1	1	1000
2	P2	2	22	100

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	Null	100

Referential Integrity Constraints Violations

~~1. Slowly Changing Dimensions.~~

~~2. Conformed Dimensions.~~

~~3. Degenerate Dimensions.~~

~~4. Junk Dimensions.~~

~~5. Role Playing Dimensions.~~

~~6. Static Dimensions.~~

~~7. Shrunk Dimensions.~~

~~8. Stacked Dimensions.~~

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. **Never Process the Fact (Ignore records NULL value for the FKs) .**
2. Park the record and retry.
3. Insert a dummy value for the missing assignment. (90% of the DW folks do this choice) .
4. Insert a record in the dimension and update (fill in the missing values) later.

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	Null	100

1. Slowly Changing Dimensions.
2. Conformed Dimensions.
3. Degenerate Dimensions.
4. Junk Dimensions.
5. Role Playing Dimensions.
6. Static Dimensions.
7. Shrunk Dimensions.
8. Stacked Dimensions.
9. **DEFERRED DIMENSION.**
10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. ~~Never Process the Fact (Ignore records NULL value for the FKs).~~
2. **Park the record and retry.**
3. Insert a dummy value for the missing assignment. (90% of the DW folks do this choice).
4. Insert a record in the dimension and update (fill in the missing values) later.

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000

Team

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12
4	Sales-2	22

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P2	2	22	4	100

LANDING AREA

1. Slowly Changing Dimensions.

2. Conformed Dimensions.

3. Degenerate Dimensions.

4. Junk Dimensions.

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. ~~Never Process the Fact (Ignore records NULL value for the FKs).~~

2. ~~Park the record and retry.~~

3. **Insert a dummy value for the missing assignment. (90% of the DW folks do this choice).**

4. Insert a record in the dimension and update (fill in the missing values) later.

Team

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12
-1	Dummy	22

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	-1	100

1. Slowly Changing Dimensions.

2. Conformed Dimensions.

3. Degenerate Dimensions.

4. Junk Dimensions.

5. Role Playing Dimensions.

6. Static Dimensions.

7. Shrunk Dimensions.

8. Stacked Dimensions.

9. **DEFERRED DIMENSION.**

10. Rapidly Changing Dimensions.

DEFERRED DIMENSION

INFERRED DIMENSIONS

FOUR solutions:

1. ~~Never Process the Fact (Ignore records NULL value for the FKs).~~

2. ~~Park the record and retry.~~

3. ~~Insert a dummy value for the missing assignment. (90% of the DW folks do this choice).~~

4. Insert a record in the dimension and update (fill in the missing values) later.

Team

TeamId	Name	EmpId
1	Sales	1
3	Manufacturing	12
4	Unknown	22

Id	ProductId	CustId	EmpId	TeamID	Amount
1	P1	1	1	1	1000
2	P2	2	22	4	100

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. ~~Deferred Dimension.~~

10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

- A dimension is a fast changing or rapidly changing dimension if one or more of its attributes in the table changes very fast and in many rows. Handling rapidly changing dimension in data warehouse is very difficult because of many performance implications.
- As you know slowly changing dimension type 2 is used to preserve the history for the changes. But the problem with type 2 is, with each change in the dimension attribute, it adds a new row to the dimension.
- If in case there are dimensions that are changing a lot, table becomes larger and may cause serious performance issues. Hence, use of the type 2 may not be the wise decision to implement the rapidly changing dimensions.

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. ~~Deferred Dimension.~~

10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

- For example: Consider patient dimension where there are 1000 rows in it. On average basis, each patient changes the 10 of attributes in a year. If you use the type 2 to manage this scenario, there will be $1000 \times 10 = 10000$ rows. Imagine if the table has 1 million rows, it'll become very hard to handle the situation with type 2. Hence, we use rapidly changing dimension approach.
- **Separate Rapidly Changing Attribute by Implementing Junk Dimension**
- Consider the fact table, in which not all the attributes of the table changes rapidly. There may be some attribute that may be changing rapidly and other not. The idea here is to separate the rapidly changing attribute from the slowly changing ones and move those attribute to another table called junk dimension and maintain the slowly changing attribute in same table. In this way, we can handle situation of increasing table size.

1. ~~Slowly Changing Dimensions.~~

2. ~~Conformed Dimensions.~~

3. ~~Degenerate Dimensions.~~

4. ~~Junk Dimensions.~~

5. ~~Role Playing Dimensions.~~

6. ~~Static Dimensions.~~

7. ~~Shrunk Dimensions.~~

8. ~~Stacked Dimensions.~~

9. ~~Deferred Dimension.~~

10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

Consider the **PATIENT** table with following attributes:

- PatientId
- Name
- Gender
- MaritalStatus
- Weight
- BMI

PATIENT
Patient_id
Name
Gender
Marital_status

PATIENT_JNK_DIM
Pat_SK
Weight
BMI

1. ~~Slowly Changing Dimensions.~~
2. ~~Conformed Dimensions.~~
3. ~~Degenerate Dimensions.~~
4. ~~Junk Dimensions.~~
5. ~~Role Playing Dimensions.~~
6. ~~Static Dimensions.~~
7. ~~Shrunk Dimensions.~~
8. ~~Stacked Dimensions.~~
9. ~~Deferred Dimension.~~
10. **RAPIDLY CHANGING DIMENSIONS.**

RAPIDLY CHANGING DIMENSIONS.

- **Link Junk Dimension and PATIENT Table:** In this step, we must link the junk dimension and patient table.
- Keep in mind; we cannot simply refer the junk dimension table by adding its primary key to patient table as foreign key.
- Because any changes made to junk dimension will have to reflect in the patient table, this obviously increases the data in patient dimension.
- Instead, we create one more table called mini dimension that acts as a bridge between Patient and Junk dimension.
- We can also add the columns such as start and end date to track the change history. Below is the structure of the mini dimension:

PATIENT
PatientId
Name
Gender
MaritalStatus



PATIENT_MINI_DIM
PatientId
PatSK
START_DATE
END_DATE



PATIENT_JN K_DIM
PatSK
Weight
BMI



PATIENT_JNK_D IM
<u>PatSK</u>
Weight
BMI
<u>StartDate</u>
<u>EndDate</u>